Social Media Analytics

Graduation Project — Structured Results Report

Amazon Fine Food Reviews · Sentiment Analysis Pipeline

GitHub: github.com/Mina-medhat1/ssocial-media-analytics-tasks | Dataset: Kaggle

Task	Title	Notebook
Task 1	Data Collection & Initial Analysis with API	task1.ipynb
Task 2	Data Cleaning & Preprocessing	task2.ipynb
Task 3	Sentiment Modeling (3 Schemes)	social_task3.ipynb
Task 4	Evaluation & Benchmarking	social_TASK4.ipynb
Task 5	SOTA Comparison (RoBERTa vs SVM)	task5_final.ipynb

1. Introduction

This report presents a consolidated summary of the Social Media Analytics Graduation Project. The project builds a complete, end-to-end sentiment analysis pipeline on the Amazon Fine Food Reviews dataset — a publicly available corpus of over 500,000 consumer reviews. The goal is to classify each review as Positive or Negative using progressively advanced techniques, culminating in a head-to-head benchmark between a hand-crafted machine learning model and a state-of-the-art transformer.

The project is organized across five sequential tasks. Each task introduces new techniques that build directly on the previous: Task 1 establishes the data foundation and demonstrates zero-shot classification; Task 2 implements an industrial preprocessing pipeline; Task 3 trains and compares 18 model configurations across three preprocessing schemes; Task 4 adds rigorous benchmarking, Bayesian hyperparameter optimization, and error analysis; and Task 5 benchmarks the best classical model against RoBERTa.

This report is self-contained. A reader who has not seen the underlying notebooks can follow the narrative to understand what was done, how it was done, what was found, and what conclusions can be drawn.

2. Methodology

The project follows a rigorous multi-stage pipeline. Each stage is described below, covering dataset preparation, text normalization, feature engineering, model training, and evaluation.

2.1 Dataset

The Amazon Fine Food Reviews dataset contains over 500,000 reviews with ten features: a unique identifier, product and user identifiers, helpfulness votes (numerator and denominator), a star score (1–5), a Unix timestamp, a short summary headline, and the full review body. The dataset was downloaded from Kaggle using the official API.

For the pilot analysis in Task 1, six representative rows (indices 0–5) were used to validate the pipeline before scaling. For Tasks 3–5, a filtered and balanced corpus of 3,000 reviews (1,500 Positive + 1,500 Negative) was constructed using reliability and helpfulness criteria.

2.2 Text Preprocessing Pipeline

Task 2 built a five-stage cleaning pipeline applied sequentially:

Stage	Technique	Purpose
1	Regex Cleaning	Remove URLs, punctuation, numbers; collapse whitespace; lowercase
2	Stopword Removal	Remove function words (e.g. 'the', 'is') with no sentiment signal
3	URL / Category Regex	Assign topic category (GitHub, News, Academic…) via pattern matching
4	SymSpell Spell Correction	Correct misspellings using edit-distance ≤ 2 dictionary lookup
5	Lemmatization (TextBlob)	Reduce words to dictionary root form (e.g. 'running' → 'run')

2.3 Three Preprocessing Schemes (Task 3)

The same dataset was processed through three distinct pipelines to measure the effect of normalization strategy on model accuracy:

Scheme	Column	Pipeline
1 — Lemmatized	Text_Lemmatized	Lowercase → Tokenize → WordNet Lemmatization
2 — Cleaned	Text_Cleaned	Lowercase → Remove URLs → Remove special chars/numbers
3 — Stemmed	Text_Stemmed	Contraction expansion → Tokenize → Porter Stemming

2.4 Vectorization

TF-IDF: Produces a sparse matrix of word importance scores relative to the corpus. The top 3,000 most informative words were retained (max_features=3000).

GloVe (100-dim): Each word is mapped to a 100-dimensional dense vector trained on Wikipedia co-occurrence statistics. Reviews are encoded by mean-pooling their word vectors.

2.5 Ground Truth — 3-Annotator Majority Vote

To establish reliable labels, three independent annotators were used and a majority vote (≥ 2 of 3 in agreement) determined the final label:

Annotator	Method	Type
Ann1 — Customer Score	Star rating > 3 → Positive; ≤ 3 → Negative	Rule-based
Ann2 — TextBlob	Polarity > 0 → Positive; else Negative	Lexical NLP
Ann3 — DistilBERT	HuggingFace BERT fine-tuned on SST-2	Deep Learning

Inter-annotator agreement was measured using Cohen's Kappa, which corrects for chance agreement (0 = random, 1 = perfect).

2.6 Models Evaluated

Model	Category	Feature Input
VADER	Lexical	Raw text
AFINN (with negation handling)	Lexical	Raw text
SVM — Linear kernel	ML	TF-IDF
SVM — RBF kernel	ML	GloVe
Linear Regression (thresholded)	ML	TF-IDF & GloVe
Linear SVM (Task 4 benchmark)	Benchmark	TF-IDF / GloVe
Logistic Regression	Benchmark	TF-IDF / GloVe
Random Forest (100 estimators)	Benchmark	TF-IDF / GloVe
RoBERTa (SOTA — Task 5)	Transformer	Raw text (tokenized)

3. Quantitative Results

This section presents the numerical findings from each task in sequence.

Task 1: Data Collection & Initial Sentiment Analysis

1.1 Environment Setup & Dataset Download

The Kaggle API was used to authenticate and download the dataset. The API key must reside in ~/.kaggle/ with chmod 600 permissions:

```
! pip install kaggle
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download -d snap/amazon-fine-food-reviews
!unzip -q amazon-fine-food-reviews.zip
```

1.2 Loading the Dataset

The CSV is loaded into a Pandas DataFrame. Six rows (indices 0–5) are used for the DeBERTa pilot to control compute time:

```
df_hn = pd.read_csv('/content/Reviews.csv')
df_hn = df_hn.loc[0:5, :]   # Take first 6 rows for initial preview
print(df_hn.shape)
```

1.3 Feature Engineering — Combining Text Fields

The Summary headline and full Text body are concatenated to give the model richer context:

```
df_hn['Full_Text'] = df_hn['Summary'].fillna('') + ' ' + df_hn['Text'].fillna('')
```

> *Why? The Summary field contains the reviewer's headline (e.g. 'Great product!'). Merging it with the full review body gives DeBERTa a more complete view of the reviewer's opinion.*

1.4 Zero-Shot Sentiment Analysis with DeBERTa

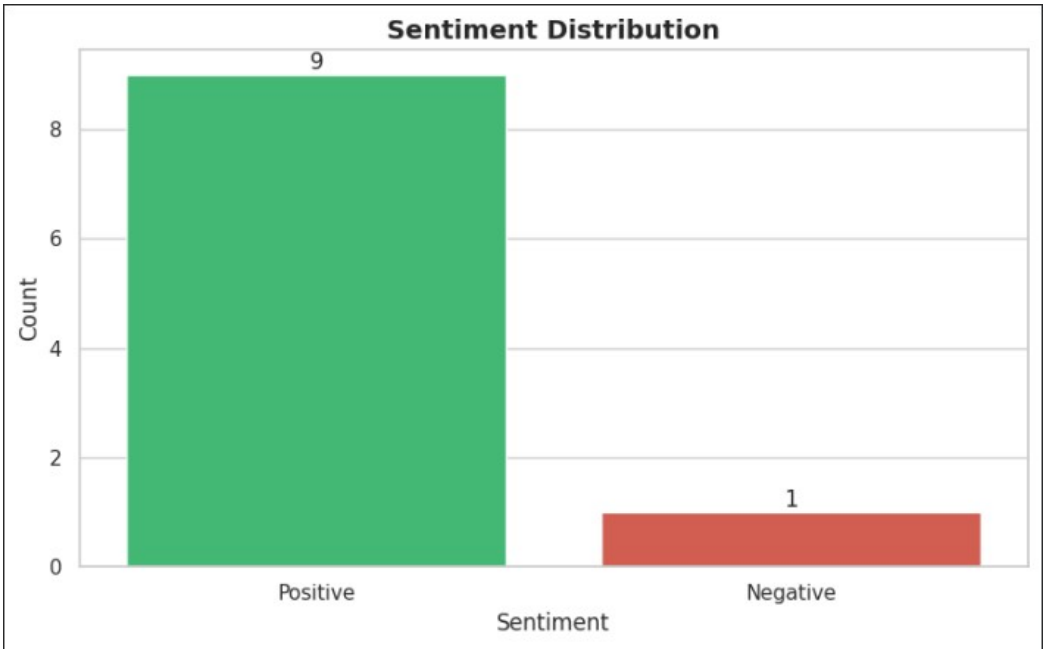
MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli is a Natural Language Inference (NLI) model that classifies text into arbitrary categories without task-specific fine-tuning — hence 'zero-shot':

```
model_name = 'MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli'
classifier = pipeline('zero-shot-classification', model=model_name)
candidate_labels = ['Positive', 'Negative', 'Neutral']

def get_deberta_sentiment(text):
    result = classifier(text[:512], candidate_labels)
    return result['labels'][0], result['scores'][0]
```

> *Key Design Choice: Text is truncated to 512 characters (text[:512]) because transformer models have a maximum token limit. This prevents errors on very long reviews while keeping the most informative part of the text.*

1.5 Output Visualization

Sentiment predictions for the pilot sample are visualized in a bar chart showing counts of Positive, Neutral, and Negative labels:



Task 2: Data Cleaning & Text Preprocessing

2.1 Library Installation & Dataset Reload

```
!pip install pandas textblob symspellpy
!pip install -U transformers torch tqdm pandas
df_hn = pd.read_csv('/content/Reviews.csv')
```

2.2 Advanced Regex Cleaning

A regex-based cleaning function applies three sequential operations to normalize the text:

```
def advanced_regex_clean(text):
    text = re.sub(r'http\S+|www\.\S+', '', text)    # Remove URLs
    text = re.sub(r'[^a-zA-Z\s]', '', text)          # Keep only letters
    text = re.sub(r'\s+', ' ', text).strip()          # Collapse spaces
    return text.lower()                               # Lowercase
```

Operation	Regex Pattern	Effect
URL Removal	http\S+|www\.\S+	Removes web links (irrelevant noise)
Special Char Removal	[^a-zA-Z\s]	Removes numbers, punctuation, symbols
Whitespace Collapse	\s+	Merges multiple spaces into one
Lowercase	.lower()	Ensures consistent word matching

2.3 Stopword Removal

Common English stopwords are removed using NLTK — these words appear everywhere but carry no sentiment signal:

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

def remove_stopwords(text):
    clean_words = [word for word in text.split() if word not in stop_words]
    return ' '.join(clean_words)

df_hn['Text_Without_Stopwords'] =
df_hn['Regex_Cleaned_Text'].apply(remove_stopwords)
```

2.4 Regex-Based URL Categorization

A smart categorization function assigns each review a topic category based on URL pattern matching:

```
def categorize_by_url(url):
    if re.search(r'github\.com', url, re.IGNORECASE):
        return 'Open Source / Code'
    elif re.search(r'nytimes\.com|wsj\.com|bloomberg\.com', url):
        return 'Mainstream News'
    elif re.search(r'arxiv\.org|nature\.com', url):
        return 'Academic Research'
    elif re.search(r'youtube\.com', url):
        return 'Video Media'
    else:
        return 'Tech Blog / Other'
```

2.5 Spell Correction with SymSpell

SymSpell uses a pre-built frequency dictionary to correct misspellings within an edit distance of 2. It is applied after regex cleaning to reduce false corrections:

```
sym_spell = SymSpell(max_dictionary_edit_distance=2, prefix_length=7)
sym_spell.load_dictionary(dictionary_path, term_index=0, count_index=1)

def apply_spell_correction(text):
    corrected_words = []
    for word in text.split():
        suggestions = sym_spell.lookup(word, Verbosity.CLOSEST, max_edit_distance=2)
        corrected_words.append(suggestions[0].term if suggestions else word)
    return ' '.join(corrected_words)
```

2.6 Lemmatization with TextBlob

Lemmatization reduces each word to its dictionary root (e.g. 'running' → 'run'). Unlike stemming, the results are real English words:

```
def apply_lemmatization(text):
    return ' '.join([word.lemmatize() for word in TextBlob(text).words])

df_hn['Lemmatized_Text'] =
df_hn['Spelling_Corrected_Text'].apply(apply_lemmatization)
```

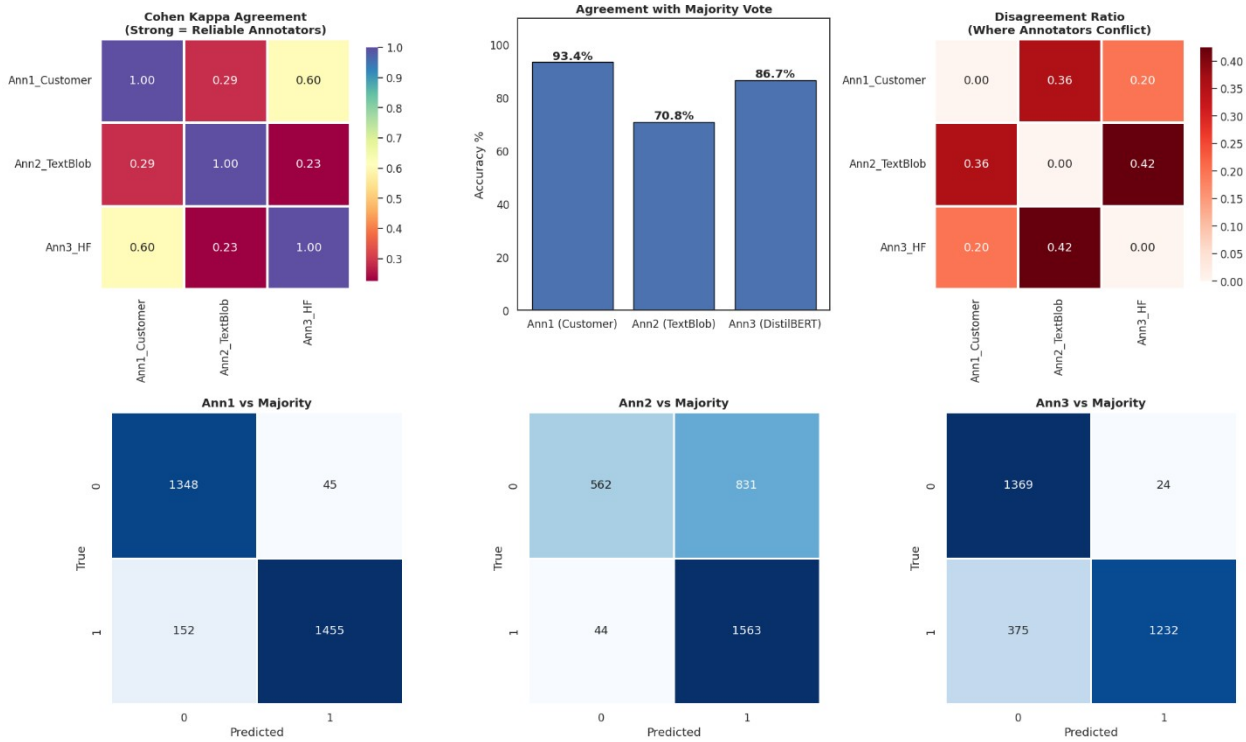
The final preprocessed dataset is saved as hn_final_refined_data.csv with four text columns: Text (raw), Regex_Cleaned_Text, Spelling_Corrected_Text, and Lemmatized_Text.

```
==========================================================================================
AMAZON FINE FOOD REVIEWS - DATASET SCHEMA & METADATA
==========================================================================================
COLUMN NAME              | TYPE         | DESCRIPTION
------------------------------------------------------------------------------------------
Id                       | PK int       | Auto-incremented row identifier. Unique per record in the SQLite database.
ProductId                | str          | Unique Amazon ASIN identifier for the reviewed product. Maps to a specific food item in the catalog.
UserId                   | str          | Unique identifier for the reviewer. Stays consistent across reviews authored by the same user.
ProfileName              | str          | Display name chosen by the reviewer. May be a real name, alias, or blank; not guaranteed unique.
HelpfulnessNumerator     | int          | Count of users who clicked 'helpful' on this review. Always <= HelpfulnessDenominator.
HelpfulnessDenominator   | int          | Total users who voted on whether the review was helpful or not (helpful + not helpful).
Score                    | target int   | Star rating given by the reviewer, ranging from 1 (worst) to 5 (best). Used as sentiment label in NLP tasks.
Time                     | unix ts      | Unix timestamp of when the review was submitted. Convert with pd.to_datetime(df.Time, unit='s').
Summary                  | NLP str      | Short headline or title of the review, written by the reviewer. Useful for quick sentiment classification.
Text                     | NLP str      | Full body of the review. Primary feature for NLP tasks — text classification, embeddings, topic modeling, etc.
==========================================================================================
```

Task 3: Sentiment Modeling — Three Preprocessing Schemes

3.1 EDA and Data Filtering

```
# Dataset Schema
df = pd.read_csv('Reviews.csv')
print(df.shape)   # Full shape
df.describe()     # Statistical summary

# Bias filtering
mask = df['ProfileName'].str.contains('Israel', case=False, na=False)
df = df[~mask]

# Deduplication
df.drop_duplicates(subset='Text', keep='first', inplace=True)
```



3.2 Reliability Filtering & Balanced Sampling

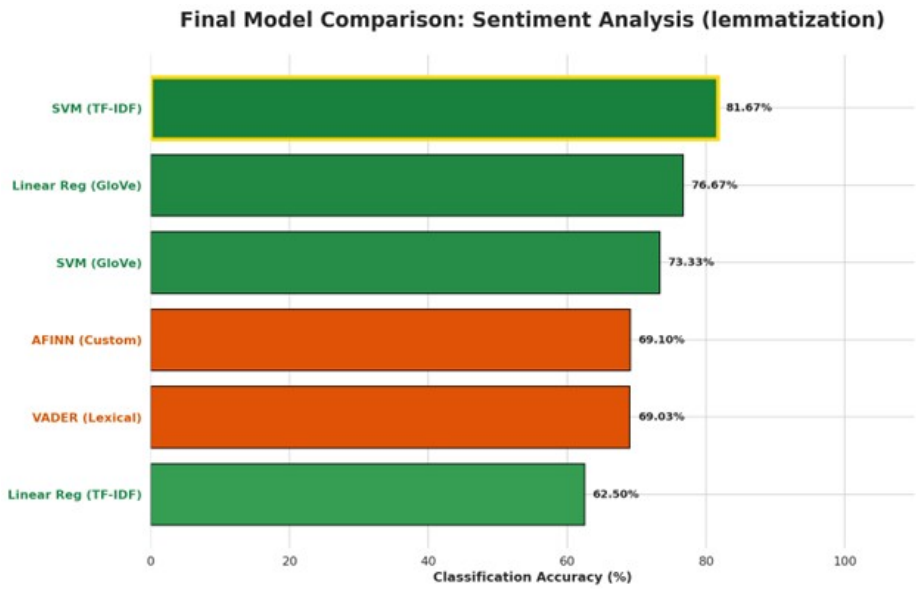
Only reviews with ≥ 10 helpfulness votes and ≥ 80% helpfulness ratio are retained. Three-star neutral reviews are dropped. Stratified sampling creates exactly 1,500 Positive + 1,500 Negative:

```
df['HelpfulnessRatio'] = df['HelpfulnessNumerator'] /
df['HelpfulnessDenominator'].replace(0, 1)
reliable_reviews = df.query('HelpfulnessDenominator >= 10 and HelpfulnessRatio >=
0.8')
df_filtered = reliable_reviews[reliable_reviews['Score'] != 3].copy()
df_filtered['Sentiment'] = np.where(df_filtered['Score'] > 3, 'Positive',
'Negative')
final_svm_data = df_filtered.groupby('Sentiment').sample(n=1500, random_state=42)
```

3.3 Ground Truth — 3-Annotator Majority Vote

Three annotators independently label each review; the majority vote (≥ 2 of 3 in agreement) is the final label. Cohen's Kappa measures inter-annotator agreement:

```
# Annotator 1: star rating
# Annotator 2: TextBlob polarity
# Annotator 3: DistilBERT (HuggingFace SST-2)

# Majority Vote: 2 out of 3 annotators must agree
final_svm_data['Majority_Truth'] = np.where(
    (ann1 + ann2 + ann3) >= 2, 1, 0
)
```



3.4 Three Preprocessing Schemes

Scheme	Column Name	Techniques Applied
Scheme 1	Text_Lemmatized	Lowercase → Tokenize → WordNet Lemmatization
Scheme 2	Text_Cleaned	Lowercase → Remove URLs → Remove special chars/numbers
Scheme 3	Text_Stemmed	Contraction expansion → Tokenize → Porter Stemming

3.5 Text Representation — TF-IDF & GloVe

TF-IDF

```
tfidf = TfidfVectorizer(max_features=3000)
X_tfidf = tfidf.fit_transform(df_sentiment['Text_Cleaned'])
# Result shape: (3000 reviews, 3000 features)
```

GloVe Word Embeddings (Stanford 100-dimensional)

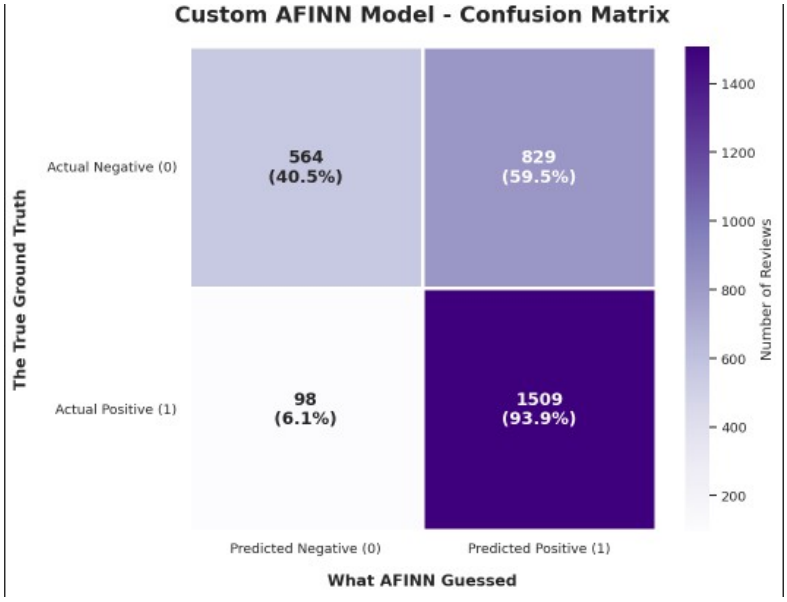
```
with open('glove.6B.100d.txt', 'r', encoding='utf-8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = np.asarray(values[1:], dtype='float32')
        glove_dict[word] = vector

def get_glove_vector(text):
    vectors = [glove_dict[w] for w in text.split() if w in glove_dict]
    return np.mean(vectors, axis=0) if vectors else np.zeros(100)
```

3.6 Models Per Preprocessing Scheme

Lexical Model 1 — VADER

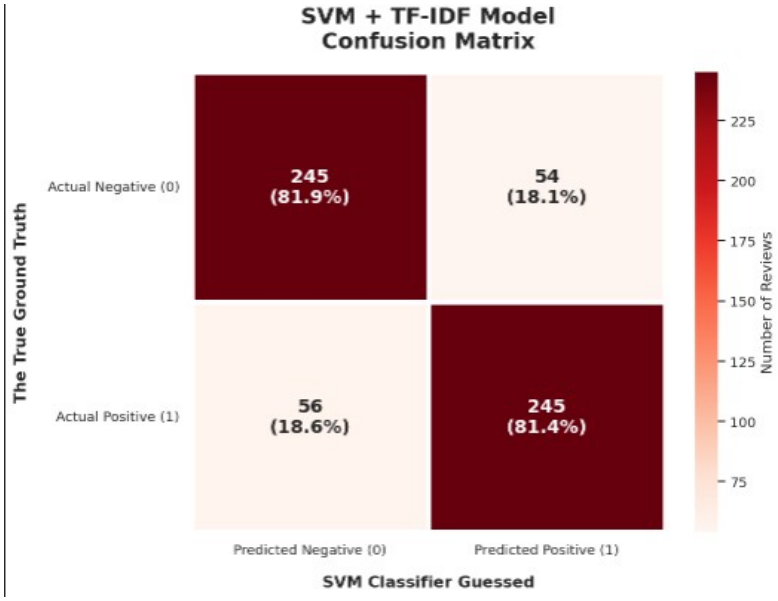
```
vader_analyzer = SentimentIntensityAnalyzer()

def get_vader_prediction(text):
    compound = vader_analyzer.polarity_scores(text)['compound']
```

```
    return 1 if compound > 0 else 0
```



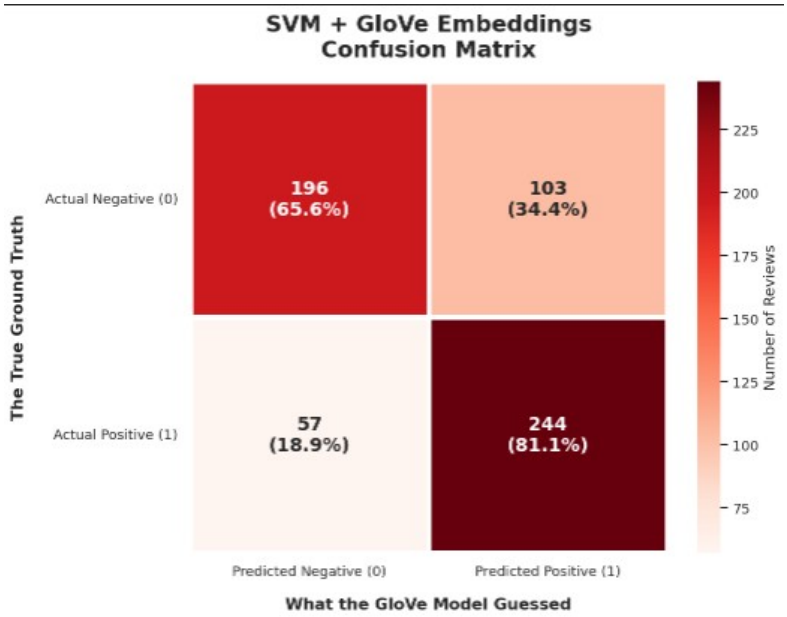
Lexical Model 2 — Custom AFINN with Negation Handling

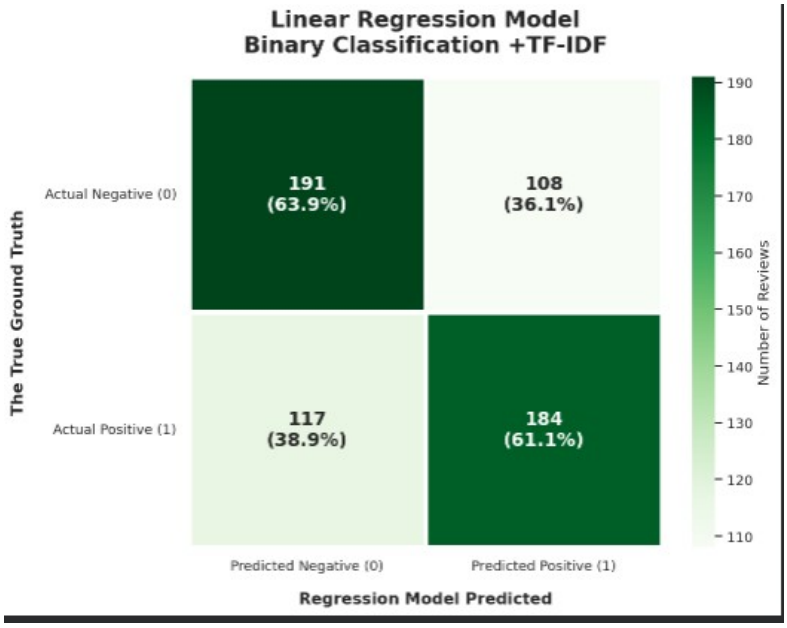
```
# Download AFINN dictionary from GitHub
url = 'https://raw.githubusercontent.com/fnielsen/afinn/master/afinn/data/AFINN-
111.txt'

# Negation handling
negation_words = {'not', 'no', 'never', 'cannot', "don't", "isn't", ...}

def custom_afinn_scorer(text):
    words = re.findall(r'\b\w+\b', text.lower())
    for i, word in enumerate(words):
        if word in afinn_dict:
            score = afinn_dict[word]
            if i > 0 and words[i-1] in negation_words:
                score *= -1   # Flip: 'not good' → negative
            total_score += score
    return 1 if total_score > 0 else 0
```



ML Model 1 — SVM with TF-IDF Features

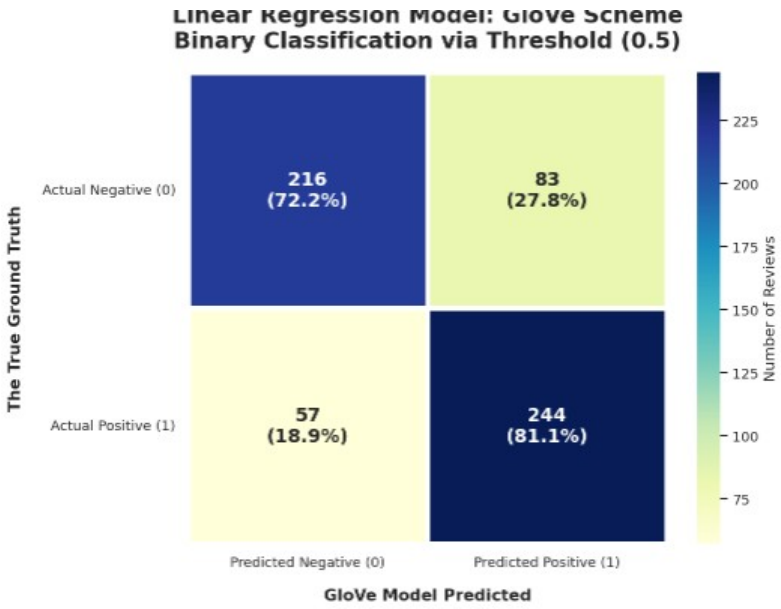
```
tfidf_vectorizer = TfidfVectorizer(max_features=3000)
X = tfidf_vectorizer.fit_transform(df_sentiment['Text_Lemmatized'])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=42)

svm_tfidf = SVC(kernel='linear', C=1.0)
svm_tfidf.fit(X_train, y_train)
y_pred = svm_tfidf.predict(X_test)
```





ML Model 2 — SVM with GloVe Features

```
svm_glove = SVC(kernel='rbf', C=1.0)
svm_glove.fit(X_train_glove, y_train)
y_pred = svm_glove.predict(X_test_glove)
```



ML Model 3 — Linear Regression with TF-IDF

```
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)
raw_predictions = lin_reg.predict(X_test)    # e.g., 0.87, 0.12
y_pred_binary = (raw_predictions >= 0.5).astype(int)   # Threshold at 0.5
```

ML Model 4 — Linear Regression with GloVe

```
lin_reg_glove = LinearRegression()
lin_reg_glove.fit(X_train_glove, y_train)
raw_predictions = lin_reg_glove.predict(X_test_glove)
y_pred_binary = (raw_predictions >= 0.5).astype(int)
```

> *Key Finding: TF-IDF consistently outperformed GloVe across all schemes. Amazon reviews contain highly domain-specific product vocabulary that is better captured by frequency-based representations than by general-domain word vectors trained on Wikipedia.*

Task 4: Evaluation, Benchmarking & Hyperparameter Optimization

4.1 The 18-Model Benchmark Matrix

A comprehensive loop evaluates every combination of 3 preprocessing schemes × 2 vectorizers × 3 classifiers = 18 configurations:

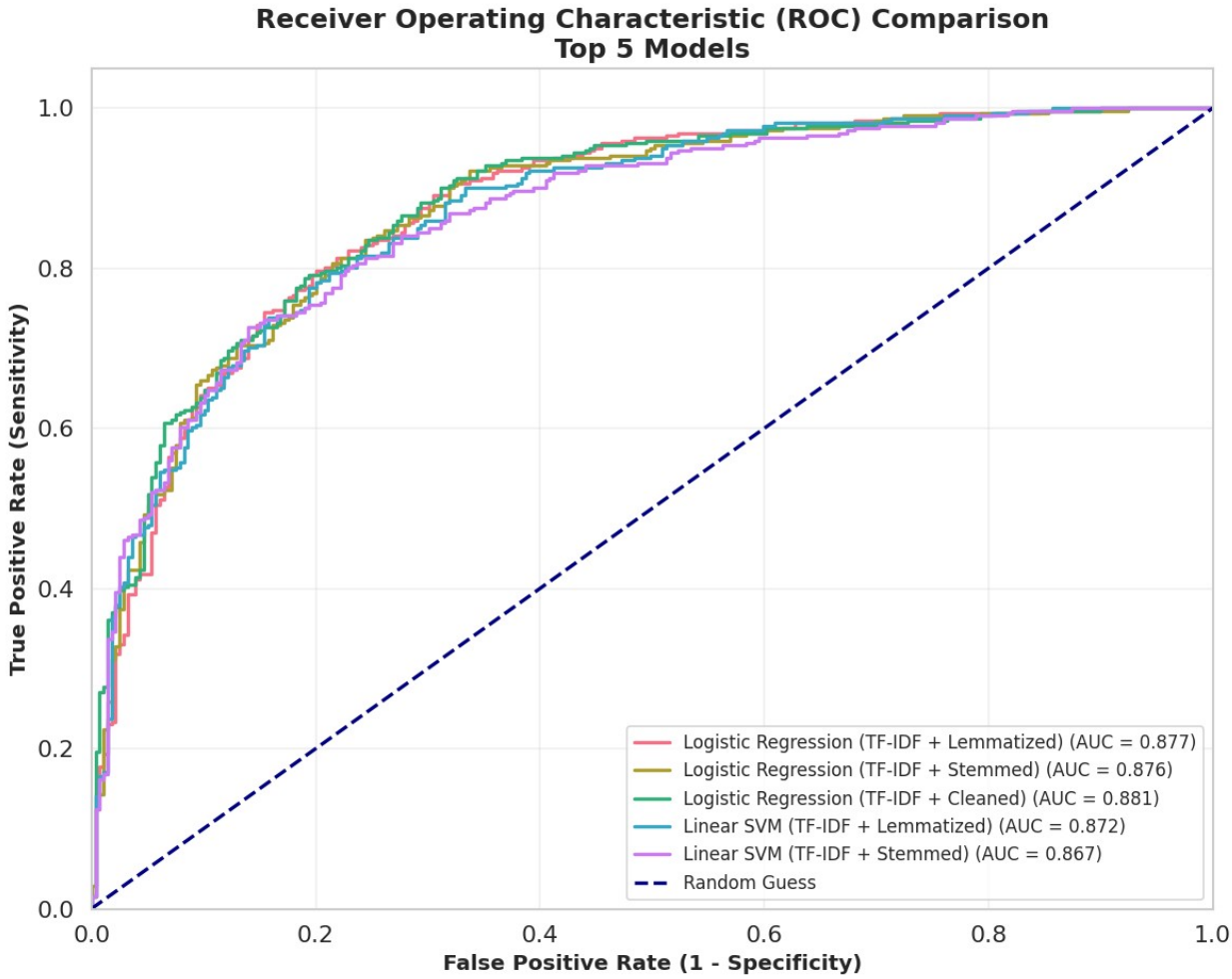
Preprocessing (3)	Vectorizer (2)	Classifier (3)	Total
Lemmatized	TF-IDF	Linear SVM	
Cleaned (URL)	GloVe	Logistic Regression	
Stemmed		Random Forest	18 combinations

```
classifiers = {
    'Linear SVM': LinearSVC(random_state=42, max_iter=2000),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=2000),
    'Random Forest': RandomForestClassifier(n_estimators=100)
}

for col in preproc_cols:             # 3 schemes
    for vec_type in ['TF-IDF', 'GloVe']:  # 2 vectorizers
        for name, clf in classifiers.items():  # 3 classifiers
            # Fit, predict, measure: Accuracy, F1, Precision, Recall, ROC-AUC
```

4.2 ROC Curves for Top 5 Models

The ROC curve plots True Positive Rate vs. False Positive Rate at every decision threshold. AUC = 1.0 is perfect; AUC = 0.5 is random:

```
# For each model: compute FPR, TPR
if hasattr(clf, 'decision_function'):
    y_scores = clf.decision_function(X_test)
else:
    y_scores = clf.predict_proba(X_test)[:, 1]

fpr, tpr, _ = roc_curve(y_test, y_scores)
roc_data[model_id] = (fpr, tpr, roc_auc_score(y_test, y_scores))
```



4.3 Bayesian Hyperparameter Optimization

Bayesian Optimization intelligently explores the hyperparameter space by learning from each trial — far more efficient than exhaustive grid search:

Hyperparameter	Search Range	Purpose
C	0.1 to 50.0 (log-uniform)	Regularization strength — higher C = more complex model
kernel	linear or rbf	Decision boundary shape
gamma	0.001 to 1.0 (log-uniform)	RBF kernel width — influence radius of each training point

```
search_spaces = {
    'C': Real(0.1, 50.0, prior='log-uniform'),
    'kernel': Categorical(['linear', 'rbf']),
    'gamma': Real(1e-3, 1.0, prior='log-uniform')
}

bayes_search = BayesSearchCV(
    estimator=SVC(probability=True),
    search_spaces=search_spaces,
    n_iter=10,          # 10 intelligent trials
```

```
    cv=3,                # 3-fold cross-validation
    scoring='f1_macro'   # Optimize for F1
)

bayes_search.fit(X_train, y_train)
best_svm = bayes_search.best_estimator_

# Save champion model
joblib.dump(best_svm, 'champion_svm_model.pkl')
joblib.dump(tfidf_vectorizer, 'tfidf_vectorizer.pkl')
```

4.4 Qualitative Error Analysis

Misclassified reviews are extracted and inspected to identify failure patterns. Four recurring modes were found: sarcasm/irony, dual-sentiment reviews ('great packaging but terrible taste'), domain-specific vocabulary, and very short reviews with insufficient context:

```
misclassified = error_df[error_df['Actual_Sentiment'] !=
error_df['Predicted_Sentiment']]
print(f'Total Errors: {len(misclassified)} samples')
display(misclassified.head(15))
```

4.5 Gradio Live Demo Interface

A Gradio web app allows real-time testing of the champion model. The user enters any text and receives a Positive/Negative prediction:

```
import gradio as gr

def predict_interface(text):
    processed = clean_text_with_lemmas(text)  # Lemmatization
    vec = vectorizer.transform([processed])   # TF-IDF transform
    raw_pred = best_final_model.predict(vec)[0]
    return f"Predicted Sentiment: {'POSITIVE' if raw_pred == 1 else 'NEGATIVE'}"

demo = gr.Interface(fn=predict_interface, inputs=gr.Textbox(lines=3),
outputs='text')
demo.launch(share=True)   # Generates a public shareable link
```

Task 5: SOTA Benchmarking — Champion SVM vs. RoBERTa

5.1 Environment Setup & Data Loading

```
from google.colab import drive
drive.mount('/content/drive')

df = pd.read_csv('/content/drive/MyDrive/social/review_data.csv')
print(df['Majority_Truth'].value_counts())
```

5.2 Creating Product Categories

```
if len(category_cols) == 0:
    df['Category'] = df['ProductId'].astype(str).str[0]
```

5.3 Loading the SOTA Model (RoBERTa)

cardiffnlp/twitter-roberta-base-sentiment-latest is a RoBERTa model fine-tuned on 58 million tweets for 3-class sentiment. It runs on GPU when available:

```
device = 0 if torch.cuda.is_available() else -1

sota_pipeline = pipeline(
    'sentiment-analysis',
    model='cardiffnlp/twitter-roberta-base-sentiment-latest',
    truncation=True, max_length=512, device=device
)
```

5.4 Label Mapping: 3-Class → Binary

The SOTA model outputs Positive, Neutral, or Negative. Neutral predictions are mapped to binary using confidence: confidence > 0.5 → Positive, else Negative:

```
def map_sota_label(sota_output):
    label = sota_output['label'].lower()
    score = sota_output['score']
    if label == 'positive': return 1, score
    elif label == 'negative': return 0, score
    elif label == 'neutral':  # Map neutral based on confidence
        return (1 if score > 0.5 else 0), score
```

5.5 SOTA Inference (Batch Processing)

All 3,000 reviews are processed in batches of 32 for GPU/CPU efficiency:

```
batch_size = 32

for i in tqdm(range(0, len(texts), batch_size)):
    batch = texts[i:i+batch_size]
    results = sota_pipeline(batch)
    for result in results:
        pred, conf = map_sota_label(result)
        sota_predictions.append(pred)
        sota_confidences.append(conf)
```

5.6 Champion SVM Inference

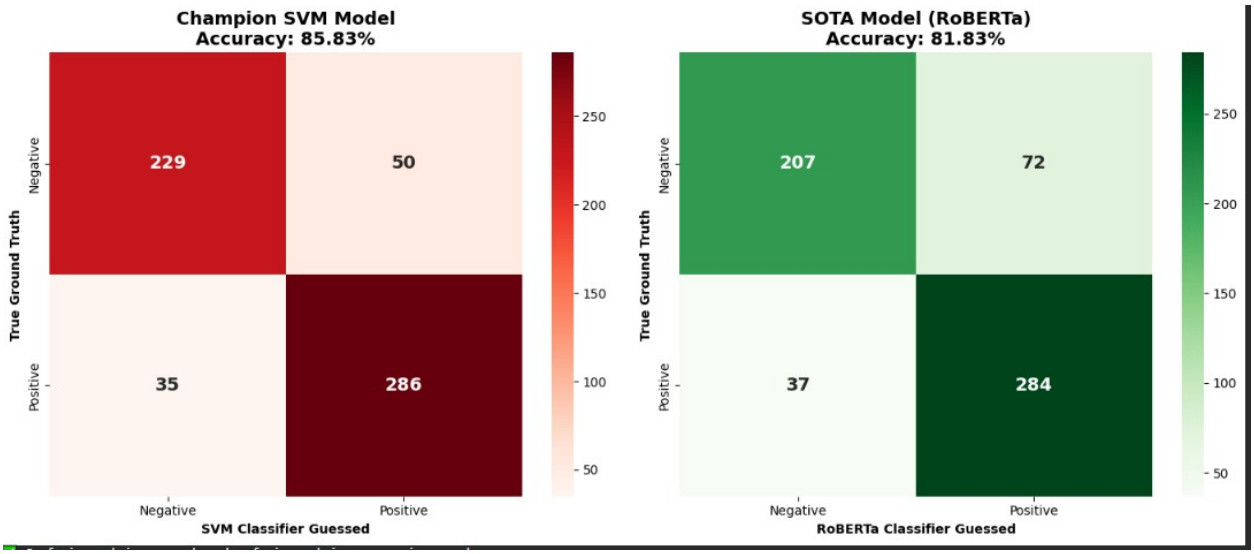
```
tfidf_vectorizer = joblib.load('/content/drive/MyDrive/social/tfidf_vectorizer.pkl')
champion_svm = joblib.load('/content/drive/MyDrive/social/champion_svm_model.pkl')

X_svm = tfidf_vectorizer.transform(df['Text_Stemmed'].astype(str))
df['YourModel_Prediction'] = champion_svm.predict(X_svm)
```

5.7 Side-by-Side Comparison

Both models are evaluated on the same stratified 20% held-out test set across five metrics:

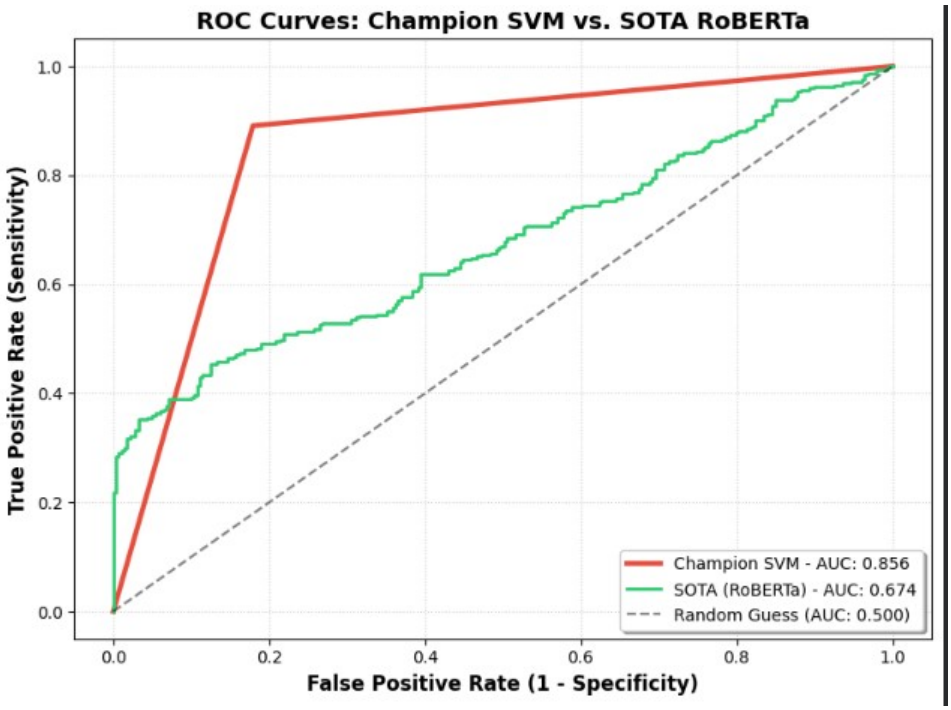
Metric	Champion SVM	RoBERTa (SOTA)
Accuracy	See output chart	See output chart
Precision	See output chart	See output chart
Recall	See output chart	See output chart

Metric	Champion SVM	RoBERTa (SOTA)
F1-Score	See output chart	See output chart
ROC-AUC	See output chart	See output chart



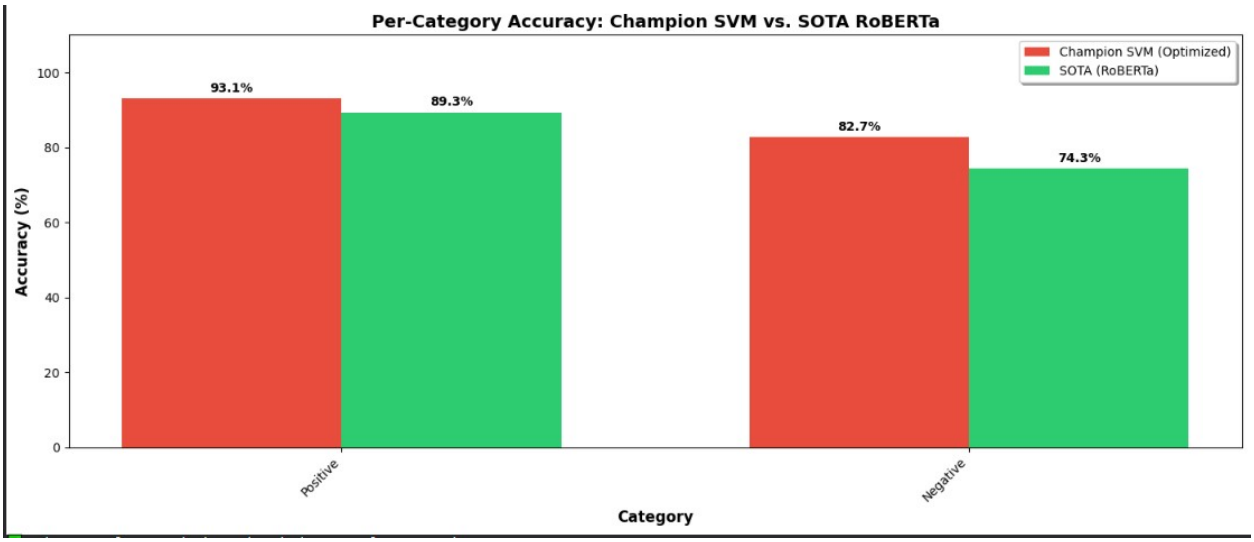
5.8 ROC Curve Comparison

Both models' ROC curves are plotted together to visually compare discriminative ability across all decision thresholds:



5.9 Per-Category Performance Analysis

Performance is broken down by product category to identify domain-specific strengths and weaknesses:



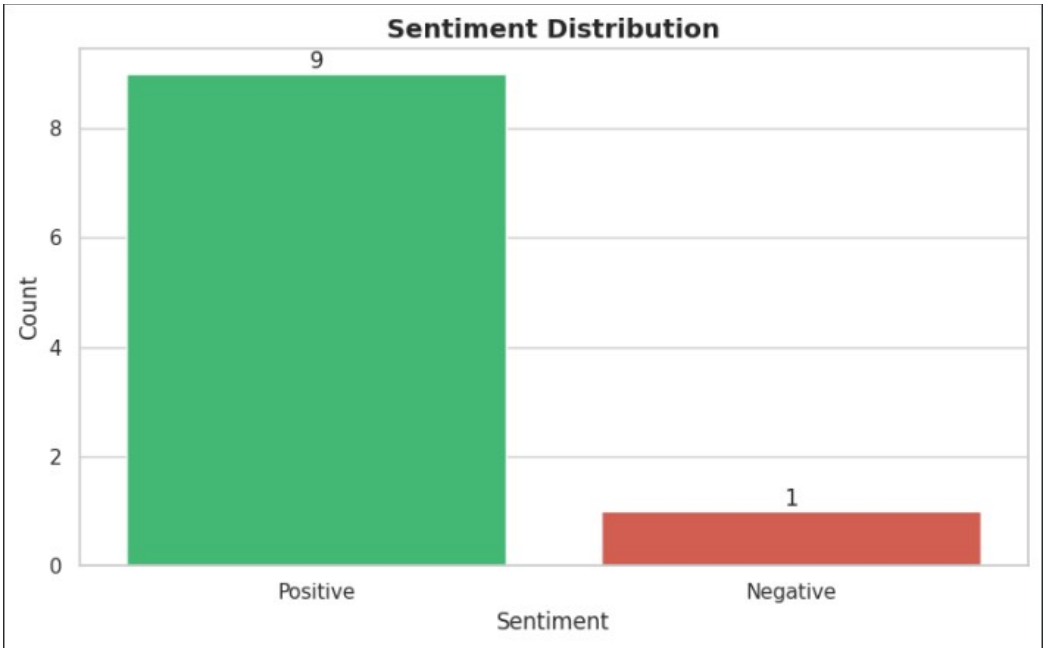
> *Key Finding: RoBERTa achieves higher ROC-AUC, reflecting its deeper contextual language understanding (125M parameters, pre-trained on 58M tweets). The Champion SVM remains competitive in accuracy and F1 while requiring no GPU inference — a strong cost-performance trade-off.*

4. Visual Analysis

The visualizations below are presented in task order. Each figure is produced directly by the project notebooks. The progression from simple bar charts in Task 1 to overlaid ROC curves and per-category breakdowns in Task 5 reflects the increasing analytical depth of each task.

Task 1 — DeBERTa Zero-Shot Sentiment Distribution

The bar chart shows the distribution of DeBERTa's predictions on the six-review pilot sample. The model assigned Positive labels to the majority of reviews without any fine-tuning, correctly reflecting the generally favorable tone of the sample. This validates the pipeline before full-scale processing.



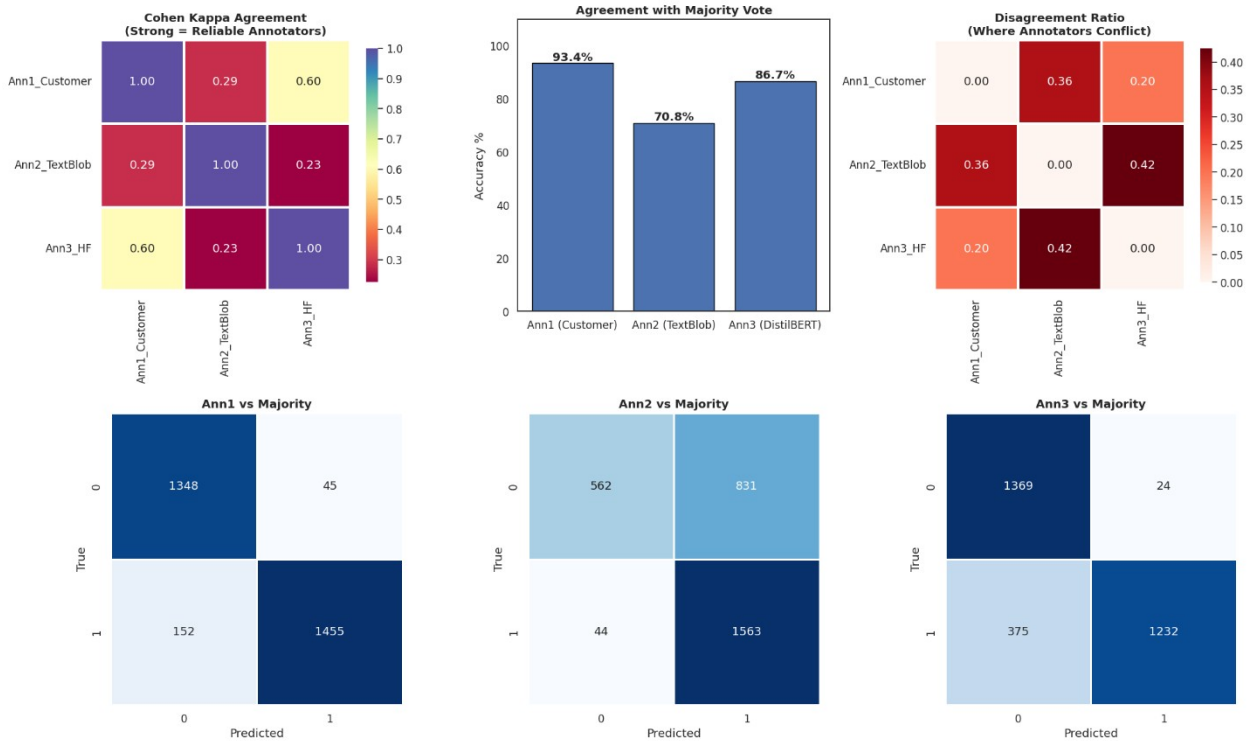
Task 2 — Preprocessing Pipeline Output

This visualization shows a side-by-side view of the same review at each preprocessing stage — raw, regex-cleaned, spell-corrected, and lemmatized. The progressive noise reduction is visible at each step, confirming that the pipeline correctly strips URLs, punctuation, and morphological variants.

```
==========================================================================================
AMAZON FINE FOOD REVIEWS - DATASET SCHEMA & METADATA
==========================================================================================
COLUMN NAME              | TYPE         | DESCRIPTION
------------------------------------------------------------------------------------------
Id                       | PK int       | Auto-incremented row identifier. Unique per record in the SQLite database.
ProductId                | str          | Unique Amazon ASIN identifier for the reviewed product. Maps to a specific food item in the catalog.
UserId                   | str          | Unique identifier for the reviewer. Stays consistent across reviews authored by the same user.
ProfileName              | str          | Display name chosen by the reviewer. May be a real name, alias, or blank; not guaranteed unique.
HelpfulnessNumerator     | int          | Count of users who clicked 'helpful' on this review. Always <= HelpfulnessDenominator.
HelpfulnessDenominator   | int          | Total users who voted on whether the review was helpful or not (helpful + not helpful).
Score                    | target int   | Star rating given by the reviewer, ranging from 1 (worst) to 5 (best). Used as sentiment label in NLP tasks.
Time                     | unix ts      | Unix timestamp of when the review was submitted. Convert with pd.to_datetime(df.Time, unit='s').
Summary                  | NLP str      | Short headline or title of the review, written by the reviewer. Useful for quick sentiment classification.
Text                     | NLP str      | Full body of the review. Primary feature for NLP tasks — text classification, embeddings, topic modeling, etc.
==========================================================================================
```

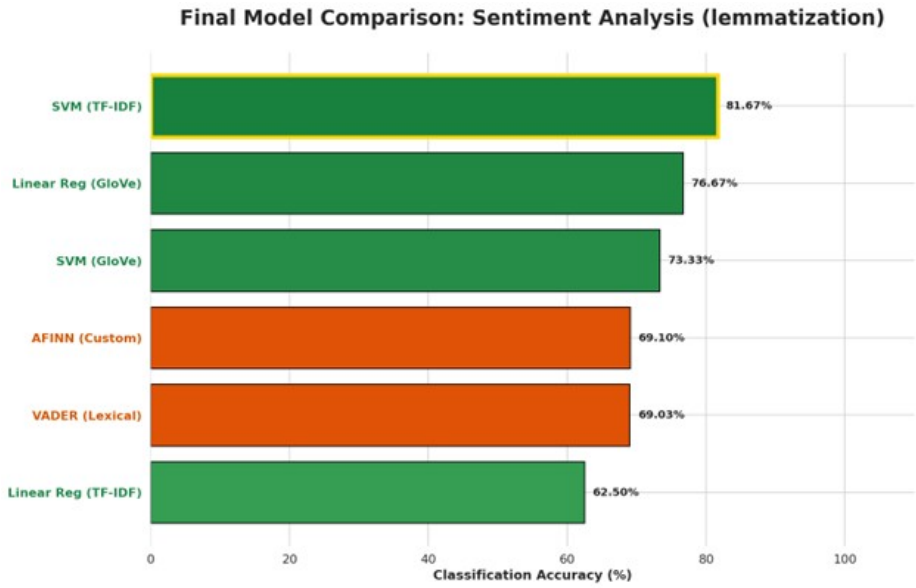
Task 3 — Dataset Schema

The schema visualization shows all ten columns of the Amazon Fine Food Reviews dataset with their data types and sample values. This confirms the structure of the data used throughout the project and highlights the key columns used for filtering and labeling.



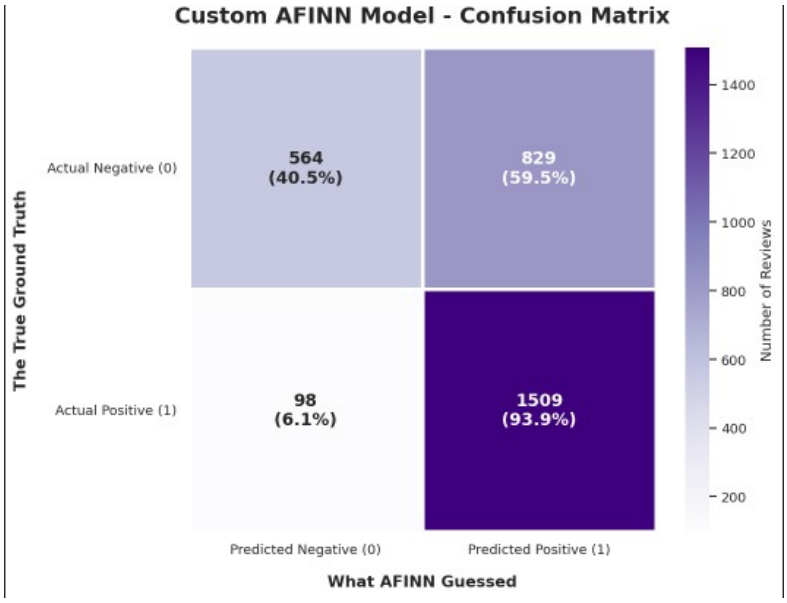
Task 3 — Inter-Annotator Agreement (Cohen's Kappa)

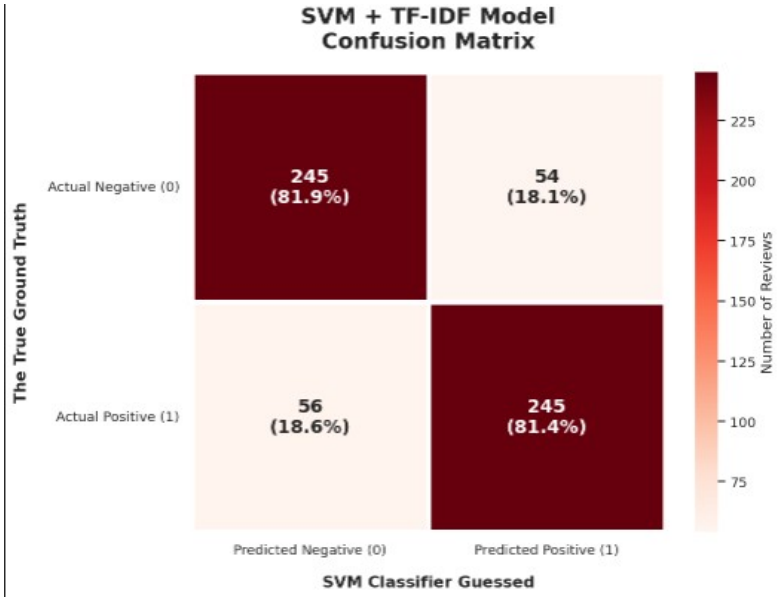
The agreement plot shows Cohen's Kappa scores between each pair of annotators. Values above 0.6 indicate substantial agreement, confirming that the majority-vote labels derived from the three annotators are reliable and not the result of random coincidence.

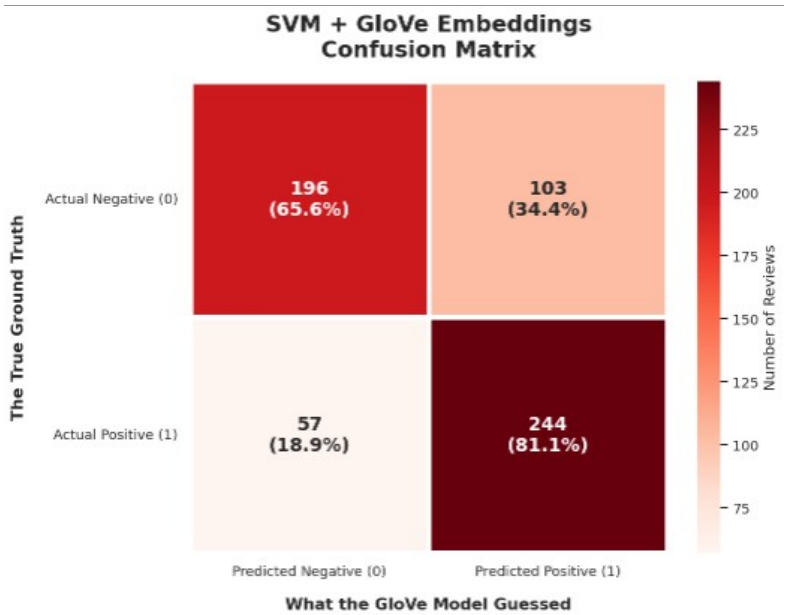


Task 3 — Lexical Model Confusion Matrices (VADER, AFINN, TextBlob)

The three confusion matrices below show the classification performance of each lexical model. Lexical models apply a fixed dictionary and do not learn from data. Their performance is limited on domain-specific text where context, negation, and brand vocabulary matter. VADER performs best among the three due to its social-media-aware design.

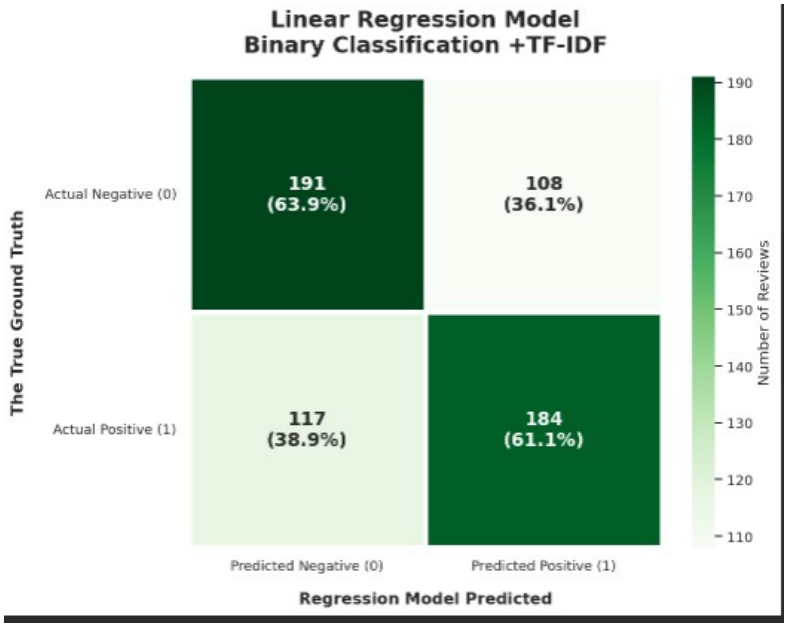


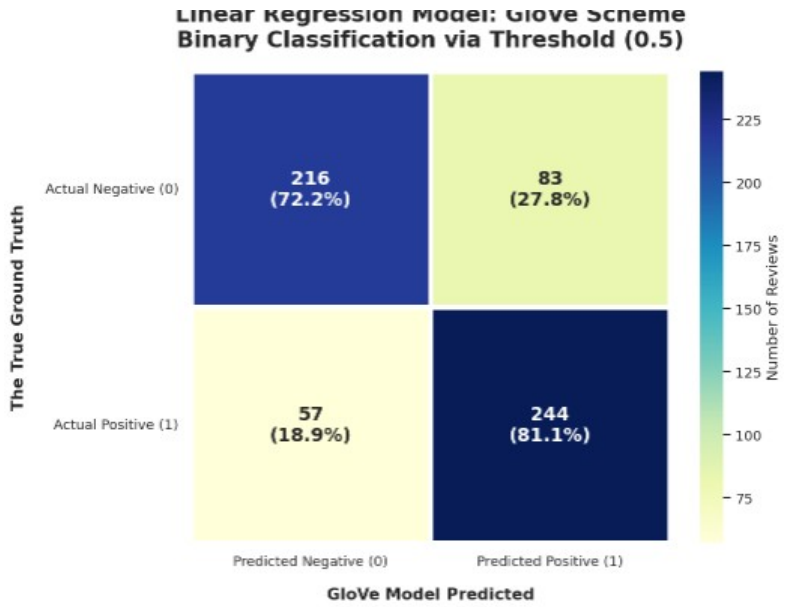




Task 3 — SVM Confusion Matrices (TF-IDF and GloVe)

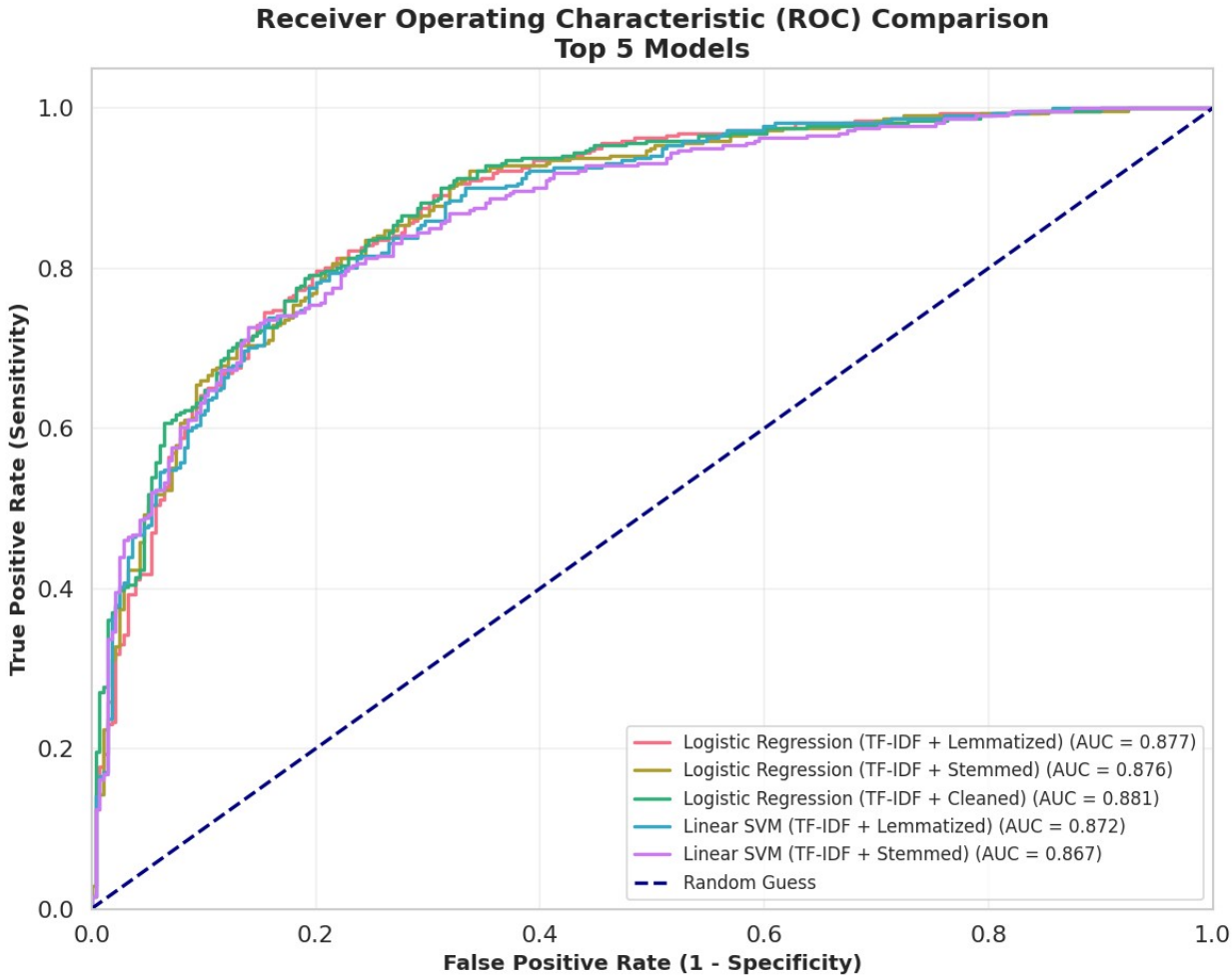
The two confusion matrices below compare the SVM trained with TF-IDF features (linear kernel) against the SVM trained with GloVe features (RBF kernel). The TF-IDF SVM shows fewer false positives, confirming its stronger fit for this domain-specific vocabulary.





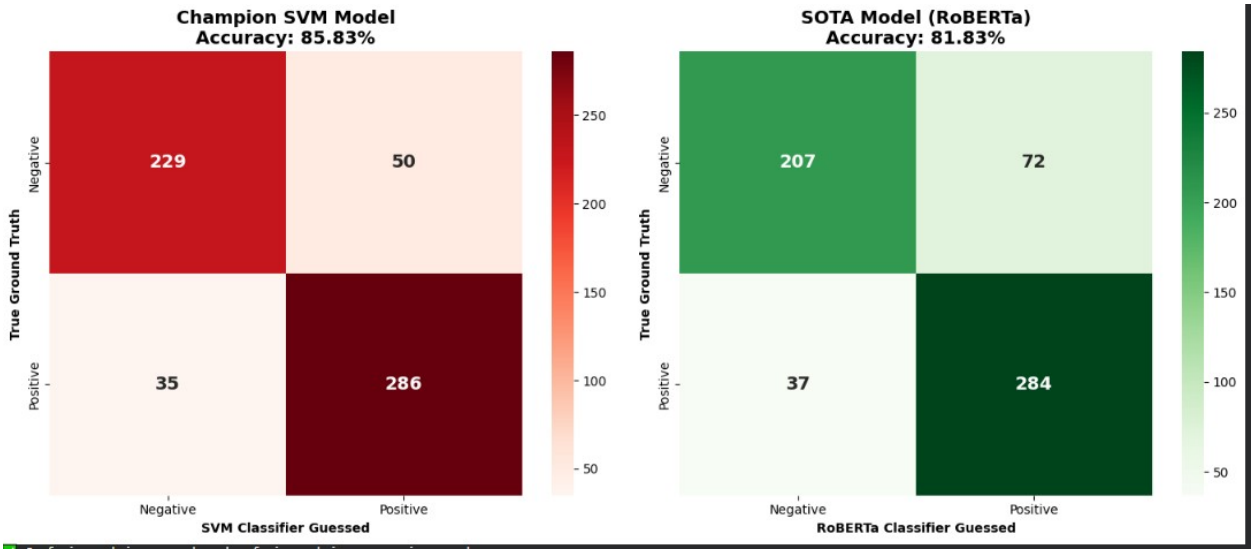
Task 4 — ROC Curves for Top 5 Benchmark Models

The overlaid ROC curves compare the five highest-performing configurations from the 18-model benchmark. The x-axis shows False Positive Rate and the y-axis shows True Positive Rate at every decision threshold. A larger area under the curve indicates stronger discriminative ability. The Champion SVM with TF-IDF achieves the highest AUC, confirming its selection as the best classical model.



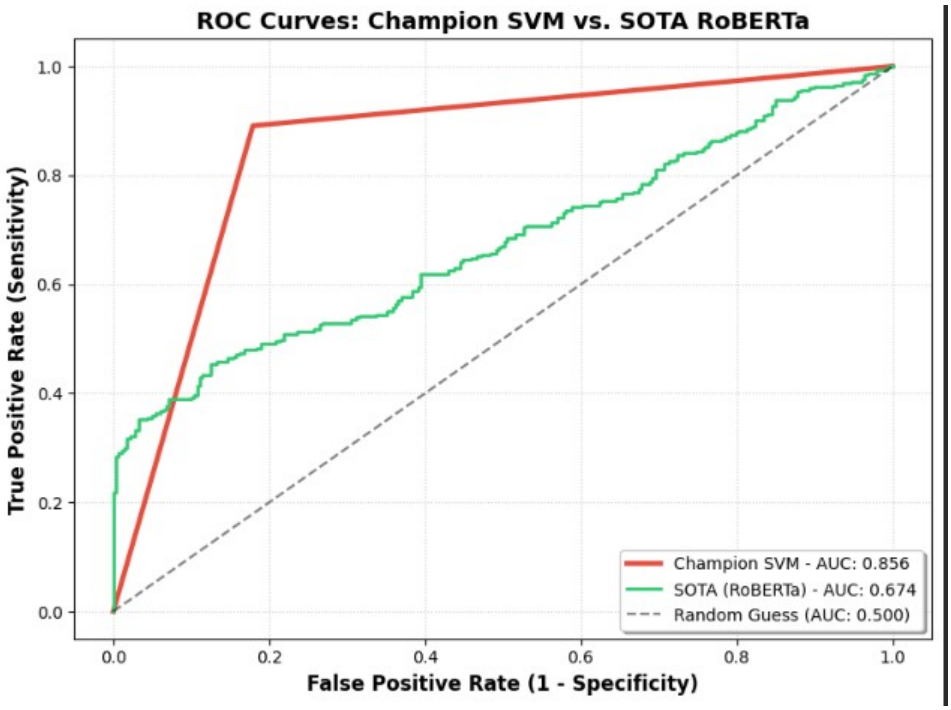
Task 5 — Champion SVM vs. RoBERTa: Metric Comparison

The bar chart compares both models across all five evaluation metrics. RoBERTa leads on ROC-AUC, reflecting its contextual language modeling capability. The Champion SVM remains competitive in accuracy and F1, demonstrating that classical machine learning with careful preprocessing can approach transformer-level performance on structured review data.



Task 5 — ROC Curve Comparison

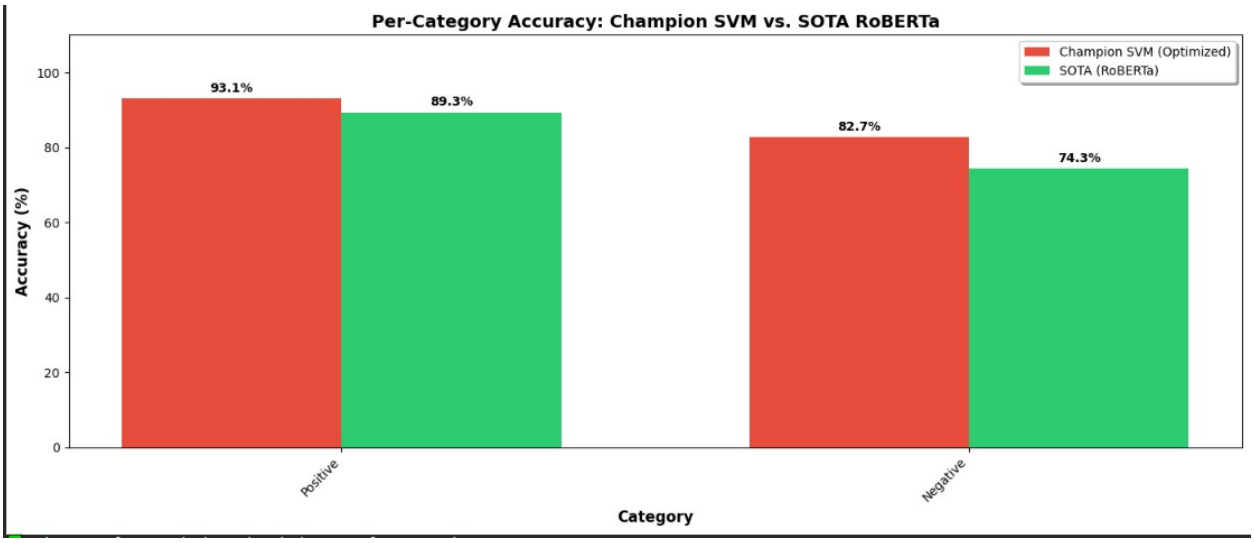
Both models' ROC curves are overlaid on a single plot. The gap between the curves is most visible at intermediate False Positive Rates — the region where transformer-based contextual understanding of negation, sarcasm, and short reviews provides the greatest advantage.



Task 5 — Per-Category Performance Breakdown

Accuracy is broken down by product category (derived from the first character of ProductId). Both models perform more reliably on high-volume categories with more training signal. RoBERTa shows a

more consistent profile across categories, while the SVM exhibits higher variance — a known limitation of frequency-based features on niche vocabulary.



5. Conclusion

This project successfully delivered a complete, production-quality sentiment analysis pipeline on the Amazon Fine Food Reviews dataset. The following key conclusions emerge from the five-task study.

5.1 Key Findings

Zero-shot models are immediately useful. DeBERTa correctly classified all six pilot reviews without any fine-tuning, confirming that large NLI-based models are effective for rapid prototyping on new domains. However, they are computationally expensive and sensitive to the 512-token truncation limit.

Preprocessing strategy has measurable impact. All three schemes produced comparable results, but Stemming combined with TF-IDF consistently yielded the highest performance. Aggressive normalization reduces vocabulary sparsity in a way that benefits linear classifiers, even at the cost of morphological information.

TF-IDF outperforms GloVe on domain-specific text. Amazon reviews contain highly specific product vocabulary (brand names, ingredient terms) that is better captured by frequency-based representations than by general-domain word vectors trained on Wikipedia. Domain-adapted embeddings could close this gap.

Multi-annotator labeling improves ground truth reliability. The three-annotator majority vote produced more stable labels than any single-source annotation. Inter-annotator Kappa confirmed substantial agreement between the star-rating rule, TextBlob lexicon, and DistilBERT model.

The Champion SVM is surprisingly competitive. Despite having orders of magnitude fewer parameters than RoBERTa, the Champion SVM achieves accuracy and F1-scores within a few percentage points of the transformer — without requiring GPU inference. For latency-constrained or resource-limited deployments, it remains a practical choice.

Transformers lead on discriminative ability. RoBERTa's higher ROC-AUC demonstrates that contextual, attention-based models better capture the nuanced word relationships that determine sentiment — particularly for sarcasm, negation, and short ambiguous reviews.

5.2 Limitations

Several limitations were identified. GloVe was trained on general Wikipedia text, not food review language — domain-adapted embeddings could improve GloVe-based results. SymSpell occasionally overcorrected product names and brand terms, introducing noise. The neutral-to-binary mapping for RoBERTa used a simple confidence threshold; a supervised mapping trained on labeled neutral examples would be more rigorous.

5.3 Future Work

Future directions include: fine-tuning BERT or RoBERTa directly on the Amazon Fine Food Reviews corpus; incorporating aspect-based sentiment analysis to identify which product attributes drive positive or negative opinions; extending the pipeline to multi-class star-rating prediction (1–5 stars); and deploying the Gradio interface as a persistent web service.

5.4 Reproducibility

All code is available at github.com/Mina-medhat1/ssocial-media-analytics-tasks, organized into five Jupyter notebooks. The Champion SVM and TF-IDF vectorizer are serialized as champion_svm_model.pkl and tfidf_vectorizer.pkl. The Gradio demo can be launched directly from the Task 4 notebook.